

Module 4 Unit testing

PyTest:

Basic asserts (test_syntax.py):

```
===== test session starts =====
platform win32 -- Python 3.10.11, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\vigne\Desktop\Cognizant_Deep_Skilling\Module4_UnitTesting\PyTest
plugins: anyio-4.11.0
collected 2 items

test_syntax.py F. [100%]

===== FAILURES =====
TestClass.test_one

self = <test_syntax.TestClass object at 0x0000020D2D024730>

    def test_one(self):
        x=0
>       assert x==1
E       assert 0 == 1

test_syntax.py:14: AssertionError
===== short test summary info =====
FAILED test_syntax.py::TestClass::test_one - assert 0 == 1
===== 1 failed, 1 passed in 0.08s =====
```

Test cases with test_***.py get executed automatically by pytest

Fixtures (test_fixtures.py) with -cov for coverage:

```
C:\Users\vigne\Desktop\Cognizant_Deep_Skilling\Module4_UnitTesting\PyTest>pytest test_fixture.py -cov
===== test session starts =====
platform win32 -- Python 3.10.11, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\vigne\Desktop\Cognizant_Deep_Skilling\Module4_UnitTesting\PyTest
configfile: ov
plugins: anyio-4.11.0
collected 3 items

test_fixture.py ... [100%]

===== 3 passed in 0.01s =====
```

Parametrization (test_parametrization.py):

```
rootdir: C:\Users\vigne\Desktop\Cognizant_Deep_Skilling\Module4_UnitTesting\PyTest
plugins: anyio-4.11.0
collected 7 items

test_parametrization.py .F.F... [100%]

===== FAILURES =====
_____ test_is_anagram[Astronomer-Moon starer] _____

phrase1 = 'Astronomer', phrase2 = 'Moon starer'

    @pytest.mark.parametrize("phrase1, phrase2", [("listen", "silent"), ("Astronomer", "Moon starer"), ("A gentleman", "Elegant man"),
        ("The Morse Code", "Here come dots!")])

    def test_is_anagram(phrase1, phrase2):
>       assert is_anagram(phrase1, phrase2)
E       AssertionError: assert False
E       + where False = is_anagram('Astronomer', 'Moon starer')

test_parametrization.py:14: AssertionError
_____ test_is_anagram[The Morse Code-Here come dots!] _____

phrase1 = 'The Morse Code', phrase2 = 'Here come dots!'

    @pytest.mark.parametrize("phrase1, phrase2", [("listen", "silent"), ("Astronomer", "Moon starer"), ("A gentleman", "Elegant man"),
        ("The Morse Code", "Here come dots!")])

    def test_is_anagram(phrase1, phrase2):
>       assert is_anagram(phrase1, phrase2)
E       AssertionError: assert False
E       + where False = is_anagram('The Morse Code', 'Here come dots!')

test_parametrization.py:14: AssertionError

===== short test summary info =====
FAILED test_parametrization.py::test_is_anagram[Astronomer-Moon starer] - AssertionError: assert False
FAILED test_parametrization.py::test_is_anagram[The Morse Code-Here come dots!] - AssertionError: assert False
===== 2 failed, 5 passed in 0.10s =====
```